

一、技术栈逐一解释

1. 阿里云 API (LLM + VLM + Embedding + Rerank 全家桶)

模型名称	用途	在本项目中用在哪里
qwen-plus	文本大语言模型	query_rewriter.py (意图识别+查询改写)、query_router.py (问题路由判断)、generator.py (最终答案生成)、metrics_generation.py (LLM Judge 评分)
qwen-vl-plus	多模态视觉语言模型	image_parser.py (图表/示意图语义理解)、generator.py (多模态问答生成)
text-embedding-v3	文本向量化	embeddings.py (文档 chunk 向量化)、multi_recall.py (查询向量化用于向量检索)
gte-rerank	重排序模型	reranker.py (对多路召回结果精排, 提升 TopK 相关性)

为什么选阿里云而非 OpenAI? 阿里云的 DashScope API 完全兼容 OpenAI SDK 格式 (base_url 指向 dashscope.aliyuncs.com/compatible-mode/v1), 所有代码直接用 from openai import OpenAI 调用, 中文场景效果更好, 且国内访问延迟低。

2. Chroma (向量数据库)

Chroma 是**嵌入式向量数据库**, 不需要单独部署服务端。数据直接持久化到本地磁盘 (data/chroma_db/)。核心概念:

- **Collection**: 类似数据库的"表", 本项目中名为 knowledge_base
- **Embedding**: 每个文本 chunk 被 text-embedding-v3 转为 1024 维向量
- **相似度检索**: 使用余弦距离 (cosine), 查询时计算 query 向量 与所有 chunk 向量的距离

3. BM25 (关键词检索算法)

BM25 是一种**基于词频的排序算法**, 属于"稀疏检索"。它的核心思想是: 查询词在文档中出现次数越多、该词在整体语料中越稀有, 则该文档得分越高。

本项目中 BM25 和 Chroma 向量检索是**互补关系**:

- BM25 擅长精确关键词匹配（如专有名词"RAG 系统"、"GPU"等），但对同义词无能为力
- Chroma 向量检索擅长语义理解（"如何加速"能匹配到"性能优化"），但可能漏掉精确匹配

4. jieba（中文分词）

BM25 需要对文本做分词处理。jieba 是 Python 最流行的中文分词库。例如输入"什么是增量索引"，jieba 输出 ["什么", "是", "增量", "索引"]，然后 BM25 基于这些词计算匹配分数。

5. RRF（Reciprocal Rank Fusion）

多路召回融合算法。公式： $RRF_score(d) = \sum 1/(k + rank_i(d))$

其中 $k=60$ ， $rank_i(d)$ 是文档 d 在第 i 路召回中的排名（从 1 开始）。特点是不依赖各路召回的原始分数（不同来源的分数不可比），只看排名，公平融合。

6. PaddleOCR（离线 OCR 引擎）

百度开源的 OCR 引擎，支持中英文混合识别。本项目用于提取图片中的文字（如论文截图、表格截图中的文字）。与 VLM 的区别：

- **OCR**：只提取文字，不关心语义，速度快，免费
- **VLM**：理解图表含义（如"这是一个柱状图，显示 2024 年销售额增长 20%"），有成本

7. RAGAS（评估框架思想）

RAGAS 是业界标准的 RAG 评估框架。本项目中 `metrics_generation.py` 实现了其核心思想（使用 LLM 作为 Judge 评估生成质量），评估三个维度：

- **Faithfulness（忠实度）**：答案是否完全来自检索到的上下文，杜绝"幻觉"
- **Answer Relevancy（答案相关性）**：答案是否切题
- **Context Precision（上下文精确度）**：检索到的内容是否有用

8. PyMuPDF (fitz)

Python 的 PDF 解析库，比 pdfplumber/PyPDF2 更快更稳定。本项目用它按页解析 PDF，获取每个文本块的字体大小、是否粗体（用于推断标题层级），以及提取嵌入的图片。

9. FastAPI + Pydantic

FastAPI 是高性能异步 Python Web 框架，Pydantic 做数据校验。4 个 API 端点：

/health、/chat、/index、/eval。

10. LangChain

主要用了两个工具类：

- RecursiveCharacterTextSplitter：递归按分隔符切分长文本
- MarkdownHeaderTextSplitter：按 # 标题层级切分 Markdown

二、逐个文件代码详解

核心基础设施层 (src/core/)

config/default.yaml — 全局配置中心

所有可调参数集中在此，修改后无需改代码。使用 \${ALIYUN_API_KEY} 环境变量占位符防止 API Key 泄露。

关键配置项：

- chunk_size: 512, chunk_overlap: 128 — 控制文档切分粒度
- top_k_bm25: 20, top_k_vector: 20 — 各路召回数量
- rrf_k: 60 — RRF 融合参数
- confidence_threshold: 0.3 — 低于此值触发二次检索
- l1_answer_ttl: 86400 — 相同问题 24 小时内直接返回缓存

src/core/config.py — 配置加载器

class Config:

```
def _resolve_env_vars(value): # 递归替换 ${VAR} 为环境变量值

def get(key_path):          # 点号路径访问, 如 config.get('index.chunk_size')
```

设计要点：

- **单例模式**：全局唯一实例，避免重复读取 YAML
- **环境变量注入**：.env 文件中的 ALIYUN_API_KEY 自动替换到配置中
- **多环境支持**：RAG_ENV=production 会加载 config/production.yaml

src/core/factory.py — 组件工厂（可插拔架构核心）

```
registry = ComponentRegistry()
```

```
# 注册新组件只需一行：
```

```
registry.register("parser", "pdf", PDFParser)
```

```
# 获取组件：
```

```
parser = registry.get("parser", "pdf")
```

这是整个系统的**解耦核心**。所有关键组件（解析器、Embedder、LLM、Reranker、Chunker）都通过注册表获取，而不是硬编码 import。换模型时只需改注册名，不改业务代码。

模块 1：离线索引层 (src/offline/)

src/offline/parsers/base.py — 解析器契约

定义了两个数据结构：

- ParsedElement：文档中的一个元素（一段文字、一个标题、一张图片、一个表格），携带页码、标题层级、所属章节
- Document：完整解析后的文档，是 ParsedElement 的列表

BaseParser 抽象类强制所有解析器实现 parse() 和 supported_extensions() 方法。

src/offline/parsers/pdf_parser.py — PDF 解析器（关键代码）

```
def _detect_heading_level(self, text, font_size, is_bold) -> int:
```

```
    # 字体 > 16pt → 一级标题
```

```
    # 字体 > 14pt + 粗体 → 二级标题
```

```
    # 字体 > 12pt + 粗体 + 短文本 → 三级标题
```

```
    # 正则匹配 "第一章"、"1.1" 等格式
```

核心逻辑：

1. PyMuPDF 逐页获取 blocks (文本块/图片块)
2. 文本块按行解析，从 spans 中提取字体大小、是否粗体
3. 字体+粗体+正则三种规则联合推断标题层级 (PDF 没有原生的标题结构，必须靠视觉特征推断)
4. 图片块提取二进制数据保存到 data/images/

src/offline/parsers/markdown_parser.py — Markdown 解析器

核心正则：匹配 # 标题

```
heading_match = re.match(r"^{1,6}\s+(.+)", line)
```

代码块用 ``` 包裹的状态机处理

表格用 | 行检测

关键设计：使用**状态机**处理代码块 (遇到 ``` 切换状态，中间内容原样保留)，而不是简单的逐行处理。

src/offline/parsers/image_parser.py — 图片解析器 (最体现多模态能力)

双阶段处理：

1. **OCR 阶段** (_run_ocr)：PaddleOCR 提取图片中的文字 → 作为文本内容索引
2. **VLM 阶段** (_run_vlm)：将图片转为 base64，调用 qwen-vl-plus，用 prompt 指导它描述图表内容 → 描述文本也参与索引

这样用户问“这个柱状图显示了什么”时，即使 query 中没有图中文字，VLM 的描述也能命中。

VLM 调用方式：

```
response = self.vlm_client.chat.completions.create(  
    model="qwen-vl-plus",  
    messages=[{"role": "user", "content": [  
        {"type": "image_url", "image_url": {"url": "data:image/png;base64,..."}},  
        {"type": "text", "text": "请详细描述这张图片的内容..."},
```

```
    ]],  
    )
```

src/offline/chunker.py — 文本切分器

核心策略两层：

1. **MarkdownHeaderTextSplitter**: 按 `####/####/####` 标题边界切分，保持章节完整性
2. **RecursiveCharacterTextSplitter**: 对过大的 chunk 二次递归拆分，分隔符优先级 `\n\n` → `\n` → `。` → `.` → 空格

每个 chunk 绑定元数据：

```
metadata = {  
    "source": "data/docs/paper.pdf", # 来源文件  
    "page": 3,                        # 所在页码  
    "section_title": "2.1 系统架构",  # 所属章节  
    "chunk_index": 5,                # 在文档中的序号  
    "doc_hash": "a1b2c3...",         # 文档 SHA256（用于增量索引）  
    "created_at": "2026-05-19T...",  # 时间戳  
}
```

src/offline/embeddings.py — Embedding 封装

```
def embed_texts(self, texts: List[str]) -> List[List[float]]:  
    # 自动分批，每批 batch_size=20 条  
    # 调用阿里云 text-embedding-v3  
    # 返回 1024 维向量列表
```

批处理的设计原因是：阿里云 Embedding API 单次最多处理 25 条文本，分批可避免超限。

src/offline/index_builder.py — 混合索引构建器（最大、最核心的文件）

三个索引合一：

- **BM25 索引**：使用 jieba 分词，中英文混合，通过 pickle 持久化到 data/bm25_index.pkl
- **Chroma 向量索引**：先调用 Embedding API 生成全部向量，再分批写入 Chroma（每批 100 条），持久化到 data/chroma_db/
- **SQLite 元数据索引**：记录每个文件的路径-哈希-索引状态，为增量索引提供依据

build_all() 是统一入口：

```
def build_all(self, documents):
```

```
    all_chunks = []
```

```
    for doc in documents:
```

```
        chunks = self.chunker.chunk_document(doc) # 切分
```

```
        all_chunks.extend(chunks)
```

```
        self.upsert_doc_meta(...) # 记录元数据
```

```
    self.build_bm25(all_chunks) # 构建关键词索引
```

```
    self.build_chroma(all_chunks) # 构建向量索引
```

模块 2：在线检索层 (src/online/)

这是 RAG 的 **7 步在线管线**，每一步都是独立的可替换组件。

Step 1 — query_rewriter.py 查询改写

```
def rewrite(self, query: str) -> RewrittenQuery:
```

```
    # 调用 LLM, system prompt 预设了 4 种意图类型
```

```
    # LLM 返回 JSON: {intent, keywords, expanded_queries}
```

```
    # 解析失败时降级为 jieba 分词规则方案
```

为什么要改写？用户输入“RAG 咋实现的”→ 改写为“RAG 系统的实现原理是什么”→ 扩展出“检索增强生成的架构设计”、“RAG 中的向量检索如何工作”等变体，多方命中检索，提高召回率。

Step 2 — query_router.py 问题路由

def route(self, query, intent) -> RouteDecision:

```
# 快速规则：含"图片/图表/如图"等词 → multimodal

# LLM 细判：text / multimodal / external 三分类

# external 类型不强制检索（比如闲聊"今天天气怎么样"）
```

Step 3 — multi_recall.py 多路召回 + RRF 融合

三路召回同时在同一个 chunk 池上检索：

召回路	方法	返回数量	特点
BM25	jieba 分词 + BM25Okapi.get_scores()	Top 20	精确关键词匹配
向量	Chroma query() 余弦距离	Top 20	语义相似度
关键词	exact-match 在 chunk 中搜索	Top 10	字符串包含匹配

RRF 融合核心代码：

```
def _rrf_merge(self, result_lists):

    for results in result_lists:

        for rank, (chunk_idx, _) in enumerate(results):

            rrf_score = 1.0 / (self.rrf_k + rank + 1) # k=60

            rrf_scores[chunk_idx] += rrf_score

    # 按总分降序排列
```

Step 4 — reranker.py 精排

def rerank(self, query, candidates):

```
# 调用阿里云 gte-rerank API

# 输入: query + 40 个候选文档

# 输出: relevance_score 排序后的 Top 10
```


API 失败时回退为 RRF 分数排序

为什么需要 Rerank? 多路召回只看 chunk 和 query 的匹配度, Rerank 会用更强大的模型重新评估**每个候选文档对回答问题的实际帮助程度**, 显著提升 TopK 质量。

Step 5 — confidence_check.py 置信度检测

```
def _compute_confidence(self, results):
```

```
    best_rrf = max(r._rrf_score for r in results)
```

```
    return min(best_rrf * 2.0, 1.0) # 归一化到 0~1
```

```
def check_and_retry(self, query, first_results, keywords):
```

```
    if confidence < 0.3: # 阈值可配
```

```
        # 用改写后的变体进行二次检索
```

```
        # 放宽关键词限制
```

```
        # 合并去重两次结果
```

这是系统的**自愈机制**: 如果第一次检索效果差 (最高分都不到 0.3), 自动用不同策略再检一次。

Step 6 — context_assembler.py 上下文拼装

```
def assemble(self, query, retrieved_chunks, top_k):
```

```
    # 取 TopK 个 chunk
```

```
    # 每个 chunk 附上引用标注: [1] p.3 §2.1 系统架构
```

```
    # 控制总长度不超过 max_context_length (4000 字符)
```

```
    # 构造最终 prompt
```

Prompt 模板控制 LLM 行为:

```
## 回答规则
```

1. 优先使用文档片段信息
2. 信息不足时明确说明

3. 使用 [来源编号] 标注引用

Step 7 — generator.py 答案生成

```
def generate(self, query, context_text, references):
```

```
    # 调用 qwen-plus 生成带引用的答案
```

```
    # 返回 GenerationResult: {answer, references, token_usage}
```

```
def generate_with_multimodal(self, query, context_text, image_paths, references):
```

```
    # 如果有图片，调用 qwen-vl-plus
```

```
    # content 中包含 text + image_url 两种类型
```

```
    # VLM 失败时降级为纯文本生成
```

模块 3：效果测评层 (src/evaluation/)

metrics_retrieval.py — 检索指标

5 个工业标准指标：

- **MRR**：第一个正确答案排在第几位？排第 1 得 1.0，排第 5 得 0.2
 - **Hit Rate@K**：前 K 个结果中至少命中一个的比例
 - **Precision@K**：前 K 个结果中正确答案占比
 - **Recall@K**：前 K 个结果召回了几成正确答案
 - **NDCG@K**：考虑排序位置的加权命中率
-

metrics_generation.py — 生成质量 (LLM Judge)

不用规则打分，而是让 **LLM 当裁判**：

Faithfulness prompt: "判断答案是否完全基于上下文..."

Relevancy prompt: "判断答案是否切题..."

Context Precision prompt: "判断上下文对回答问题是否有帮助..."

每个维度调用一次 LLM，输出 0-1 分数。

ablation_runner.py — 消融实验

对比不同策略组合的效果，输出 Markdown 报告到 reports/:

- 切分策略对比: chunk_size=256/512/1024
 - 召回策略对比: 纯向量 vs 纯 BM25 vs 混合
 - Rerank 对比: 无 Rerank vs gte-rerank
-

模块 4: 工程优化层 (src/engineering/)

incremental_index.py — 增量索引

```
def detect_changes(self):
```

```
    current_hashes = {path: sha256(path)} # 扫描当前文档
```

```
    indexed = sqlite.query("path, hash") # 查询已索引的
```

```
    # 对比差异 → 新增/修改/删除 三类
```

处理逻辑:

- **新增文件**: 解析 → 写入 Chunks → 同步到 BM25 + Chroma
- **内容变更**: 删除旧 chunks → 重新索引
- **文件删除**: 标记为 deleted → 从 BM25 移除对应 chunks 并重建

为什么 BM25 需要重建? BM25Okapi 不支持直接删除单个文档，删除后必须从剩余 chunks 重建索引。但 Chroma 原生支持 collection.delete(ids) 单个删除。

cache_manager.py — 三层缓存

L1(答案缓存): query MD5 → {answer, references}, TTL=24h

L2(检索结果): query MD5 → {recalled_docs}, TTL=1h

L3(Embedding): text MD5 → [0.123, -0.456, ...], TTL=永久

全部基于 SQLite 实现，零额外依赖。Key 使用 MD5 哈希避免存储明文 query。

file_watcher.py — 文件监听

基于 watchdog 库，监控 data/docs/ 目录：

- 检测到文件变更 → 2 秒防抖（避免编辑器保存产生多次事件）→ 自动触发增量索引
- 后台线程运行，不阻塞主流程

API 服务层 (src/api/)

routes.py 中的 RAGService 类是整个系统的**服务编排器**，串联了所有模块。

chat() 方法是核心流程的完整实现：

1. L1 缓存检查 → 命中直接返回
2. query_rewriter.rewrite() → 意图+关键词+变体
3. query_router.route() → 判断走文本/多模态
4. multi_recall.recall() → 三路召回+RRF 融合
5. reranker.rerank() → 精排 TopK
6. confidence_checker.check() → 低分则二次检索
7. context_assembler.assemble() → 拼装 prompt
8. generator.generate() → LLM 生成答案
9. cache.set() → 写入 L1 缓存

三、运行文档

环境准备

```
cd E:\机器学习\git_rag
```

```
pip install -r requirements.txt
```

注意： PaddleOCR 首次运行自动下载模型（~100MB），需要网络。

API Key 配置

已在 .env 文件中:

```
ALIYUN_API_KEY=sk-6e96117b65e04d92bdbed0bd5ceba5f3
```

准备知识库文档

把 PDF、Markdown、图片放入 data/docs/:

data/docs/

├── 论文笔记.md

├── 技术文档.pdf

├── 架构图.png

└── ...

构建索引（必须第一步）

增量索引（日常使用）

```
python cli.py index --path ./data/docs
```

全量重建（修改了切分配置后）

```
python cli.py index --path ./data/docs --full
```

交互式问答

```
python cli.py chat
```

```
>>> 什么是 RRF 融合算法？
```

```
[意图: knowledge_qa]
```

```
[路由: text]
```

RRF (Reciprocal Rank Fusion) 是一种多路召回融合算法...

```
[1] engineering_notes.md (p.N/A, §增量索引策略)
```

运行评估

```
python cli.py eval --dataset ./data/eval_queries.json
```

启动 API 服务

```
python cli.py serve --port 8000
```

访问 <http://localhost:8000/docs> 查看 Swagger 文档

```
curl -X POST http://localhost:8000/api/v1/chat \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"query": "什么是混合检索?", "top_k": 10}'
```

文件监听自动索引

```
python cli.py watch --path ./data/docs
```

常见问题

问题	解决方法
import fitz 失败	pip uninstall PyMuPDF -y && pip install PyMuPDF
PaddleOCR 首次很慢	正常的，在下载模型（~100MB），之后会快
Chroma 索引损坏	rm -rf data/chroma_db data/bm25_index.pkl data/metadata.db 后重建
API 调用超时	检查网络 + API Key 有效性
中文分词效果差	确保 pip install jieba

总结：42 个文件，4 大模块，7 步检索管线，3 层缓存，2 种索引，1 套完整的多模态 RAG 系统。